

11. VARIOS

Ofrecer la descarga de un archivo desde PHP

Podemos enviar a un cliente no sólo HTML, sino otro tipo de contenido y utilizar la función `readfile` para enviársela.

Los navegadores leen el tipo MIME de las cabeceras HTTP para saber cómo deben mostrar los contenidos. Mediante PHP indicamos el tipo MIME

- `Content-type: application/pdf` : podremos enviar un documento PDF, es válido para cualquier extensión
- `Content-type: application/octet-stream`: le decimos que enviaremos un paquete pero no sabe cuál.
- `Content-type: application/force-download`: le decimos que tiene que descargar el archivo, no abrirlo

```
header('Content-type: application/pdf')
```

Además, usaremos la cabecera `Content-Disposition: attachment;filename="nombre"` para indicar el archivo que descargaremos

```
$enlace = $_GET['id'];
```

```
header ("Content-Disposition: attachment; filename=$enlace ");
```

Por ultimo usaremos la función `readfile` para enviar el archivo

```
readfile($enlace);
```

Podemos indicar el tamaño que tendrá el archivo

```
header ("Content-Length: ".filesize($enlace));
```

El enlace: `<a href="descarga.php?file=archivo.zip">Descargar archivo</a>`

Evidentemente, este código es muy básico. Hay muchas cosas que debemos controlar:

- Que exista el archivo que queremos descargar
- Que no nos modifiquen la ruta del archivo vía URL. Podemos utilizar la función `basename()` para obtener sólo el nombre del archivo de la ruta pasada
- Que sólo se descargue el archivo solicitado

Podemos utilizar funciones de php para generar en tiempo de ejecución los valores de las cabeceras

Subida de archivos

Una actividad bastante común en un sitio web es almacenar un archivo en el servidor: subir fotos, documentos, programas, etc.

Para poder llevar a cabo la subida de un archivo, es necesario tener un formulario con atributo `enctype="multipart/form-data"`.

Utilizaremos la colección `$_FILES` para este proceso, dicho array asociativo contendrá toda la información de los archivos a subir

`$_FILES['fichero_usuario']['name']`: El nombre original del fichero en la máquina del cliente.

`$_FILES['fichero_usuario']['type']`: El tipo del fichero, si el navegador lo proporciona. Esto no se comprueba desde php

`$_FILES['fichero_usuario']['size']`: El tamaño, en bytes, del fichero subido.

`$_FILES['fichero_usuario']['tmp_name']`: El nombre temporal del fichero en el cual se almacena el fichero subido en el servidor.

`$_FILES['fichero_usuario']['error']`: El código de error asociado a esta subida.

Para limitar el tamaño de los archivos del lado del cliente, se coloca delante del control `<input type='file' />` un campo oculto con el `name=MAX_FILE_SIZE`. Su valor es el tamaño, en bytes, máximo que admitimos. También se puede configurar en `php.ini`

Los archivos enviados a través de un formulario se almacenan en una carpeta temporal del servidor con un nombre provisional, se puede configurar la ubicación en el `php.ini`, (`upload_temp_dir`). A menos que los movamos a otra ubicación, se borrarán en cuanto termine la ejecución del script. Para mover estos archivos de su almacenamiento temporal a otro definitivo se utiliza la función `move_uploaded_file(origen, destino)`.

Si existe en esa ubicación otro archivo con el mismo nombre, se sobrescribe.

El directorio temporal al que se suben los archivos se configura mediante la directiva `upload_tmp_dir`, que en el caso de WAMP contiene el valor `c:/wamp/tmp`.

Otras directivas de `php.ini` que afectan a la subida de archivos son:

- `file_uploads`: Determina si se permite o no la subida de archivos a través de formularios.
- `upload_max_filesize`: Establece el tamaño máximo de los archivos que se pueden subir.
- `post_max_size`: Establece el tamaño máximo de los datos POST, incluidos los archivos. Este valor debería ser mayor que el anterior.

Una vez subido el archivo a través del formulario podremos capturarlo en PHP a través de `$_FILES` y acceder a los siguientes datos:

```
<input type="file" name="foto">
```

- name: El nombre original del archivo en el ordenador del cliente.

```
$_FILES['foto']['name']
```

- type: El tipo MIME declarado del archivo. Tenga en cuenta que un usuario malintencionado podría falsear la declaración de tipo MIME y hacernos llegar un script camuflado como una imagen, por ejemplo. Conviene comprobar al menos la extensión; y para tener mayor seguridad puede recurrirse al módulo FileInfo de PHP (<http://www.php.net/manual/es/book.fileinfo.php>).

```
$_FILES['foto']['type']
```

- size: El tamaño en bytes del archivo.

```
$_FILES['foto']['size']
```

- tmp_name: La ruta completa del archivo temporal en el servidor. Puede usarse como origen de `move_uploaded_file`.

```
$_FILES['foto']['tmp_name']
```

- error: Si se produce algún error durante la subida del archivo, su código se almacenará en esta variable. Puede consultar los códigos de los errores en la dirección <http://www.php.net/manual/es/features.file-upload.errors.php>.

Generar contenidos diferentes a páginas HTML con PHP

Podemos generar imágenes, documentos PDF,... en tiempo real, sin tener que leerlos de ningún archivo y enviarlos al cliente. Para ello utilizamos librerías de PHP como GD, PDFlib, Ming,...

Algunas de estas funciones (GD)

- `imageCreate(ancho, alto)`: retorna una referencia a la imagen que vamos a crear con esas dimensiones, esa referencia la utilizaremos en las demás funciones
- `imagecreatefromjpg`: crea una imagen a partir de un fichero o URL, por ejemplo nos permitirá utilizar como fondo una imagen JPG. Retorna una referencia a la imagen creada.
- `Imagecolorallocate(imagen,rojo,verde,azul)`: Sirve para incluir colores en la paleta de la imagen.
- `imageFill(imagen,x,y,color obtenido con la anterior)`: rellena con color a partir de las coordenadas.

- `imageString(imagen,tamañoFuente,x,y,valor,color)`
- `imageTTFtext`: Sirve para escribir caracteres en la imagen usando fuentes TrueType.
- `imageLine(imagen,x_inicio,y_inicio,x_fin,y_fin)`: dibuja una línea en la imagen con esas coordenadas
- `imageJpg`: Envía la imagen al agente de usuario.
- `imageDestory`: Borra la imagen de la memoria del servidor.

Si queremos incorporar esta imagen a una página HTML, incluimos una llamada al php que lo genera

```
<imgsrc="archivo.php" >
```

Envío de Correos

En PHP hay una función llamada `mail()` que permite configurar y enviar mensajes de correo. Recibe tres parámetros de manera obligada y otros dos opcionales. Devuelve `true` si se envió el mensaje correctamente y `false` en caso contrario.

Parámetros necesarios en todos los casos

Destinatario: la dirección de correo o direcciones de correo que han de recibir el mensaje. Si incluimos varias direcciones debemos separarlas por una coma.

Asunto: para indicar una cadena de caracteres que queremos que sea el asunto del correo electrónico a enviar.

Cuerpo: el cuerpo del mensaje, lo que queremos que tenga escrito el correo.

Ejemplo de envío de un mail sencillo

```
<?php
$destinatario = "pepito@desarrolloweb.com";
$asunto = "Este mensaje es de prueba";
$cuerpo = '
<html>
<head>
    <title>Prueba de correo</title>
</head>
<body>
<h1>Hola amigos!</h1>
<p>
<b>Bienvenidos a mi correo electrónico de prueba</b>. Estoy
encantado de tener tantos lectores. Este cuerpo del mensaje
es del artículo de envío de mails por PHP. Habría que
cambiarlo para poner tu propio cuerpo. Por cierto, cambia
también las cabeceras del mensaje.
</p>
```

```
</body>
</html>
';

¿>
```

Parámetros opcionales del envío de correo

Headers: Cabeceras del correo. Datos como la dirección de respuesta, las posibles direcciones que recibirán copia del mensaje, las direcciones que recibirán copia oculta, si el correo está en formato HTML, etc.

```
<?php
$destinatario = "pepito@desarrolloweb.com";
$asunto = "Este mensaje es de prueba";
$cuerpo = '
<html>
<head>
    <title>Prueba de correo</title>
</head>
<body>
<h1>Hola amigos!</h1>
<p>
<b>Bienvenidos a mi correo electrónico de prueba</b>.
Estoy encantado de tener tantos lectores. Este cuerpo del
mensaje es del artículo de envío de mails por PHP. Habría
que cambiarlo para poner tu propio cuerpo. Por cierto,
cambia también las cabeceras del mensaje.
</p>
</body>
</html>
';
```

//para el envío en formato HTML

```
$headers = "MIME-Version: 1.0\r\n";
```

```
$headers .= "Content-type: text/html; charset=iso-8859-1\r\n";
```

//dirección del remitente

```
$headers .= "From: Miguel Angel Alvarez<pepito@desarrolloweb.com>\r\n";
```

//dirección de respuesta, si queremos que sea distinta que la del remitente

```
$headers .= "Reply-To: mariano@desarrolloweb.com\r\n";
```

//ruta del mensaje desde origen a destino

```
$headers .= "Return-path: holahola@desarrolloweb.com\r\n";
```

//direcciones que recibían copia

```
$headers .= "Cc: maria@desarrolloweb.com\r\n";
```

```
//direcciones que recibirán copia oculta
$headers .= "Bcc: pepe@pepe.com,juan@juan.com\r\n";

mail($destinatario,$asunto,$cuerpo,$headers)
?>
```

Sentencias namespace , use

Los espacios de nombres son una de las utilidades que han aparecido en PHP 5, Sirven para organizar el código, de manera que los nombres que nosotros reservemos a la hora de crear clases o funciones no entren en conflicto con los que hayan podido, o puedan en el futuro, crear otras personas.

Hasta la creación de los namespaces, para evitar colisiones de nombres, los desarrolladores estaban obligados a ser un poco imaginativos con los nombres que utilizaban, creando nombres extra largos para sus clases, evitando así que otras personas en el futuro pudieran usar esos mismos nombres.

Obviamente, esa solución no era la mejor, así que PHP incorporó los espacios de nombres, que nos permiten reservar nombres para las funciones, clases, interfaces, constantes, etc, que solo tienen validez en cierto namespace. Al crear una librería, colocarías tus funciones dentro de un espacio de nombres, de manera que otros desarrolladores puedan usar los mismos nombres sin que colisionen, por estar en distintos espacios de nombres.

Para declarar un espacio de nombre debes poner como primera instrucción del archivo (antes incluso del html)

namespace nombre;

Para utilizar las clases, funciones,... declaradas en dicho archivo además de incluirlo debes usar la sentencia use

use nombre_del_namespace\clase;